

Ankur Bhardwaj

PCSE03_Student

 Quick Submit Quick Submit KIET Group of Institutions, Ghaziabad

Document Details

Submission ID**trn:oid::1:3457180104****Submission Date****Jan 14, 2026, 9:28 AM GMT+5:30****Download Date****Jan 14, 2026, 9:32 AM GMT+5:30****File Name****Updated_Research_Paper_-_PCSE-03.pdf****File Size****1.7 MB****7 Pages****3,258 Words****19,432 Characters**

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



JARVIS:AI Powered Desktop Assistant

Intelligent Automation and Real-Time

Human–Computer Interaction

Aditi Agrawal

Department of CSE

KIET Group of Institutions

Ghaziabad, India

aditi.2226cse1066@kiet.edu

Abha Gangwar

Department of CSE

KIET Group of Institutions

Ghaziabad, India

abha.2226cse1148@kiet.edu

Ekta

Department of CSE

KIET Group of Institutions

Ghaziabad, India

ekta.2226cse1019@kiet.edu

Dr. Ankur Bhardwaj

Department of CSE

KIET Group of Institutions

Ghaziabad, India

ankur.bhardwaj@kiet.edu

Abstract—With the ever-rising need for intelligence in more natural interactions between humans and computers, virtual assistants are increasingly making their mark—and now have the ability to decode everyday conversations and perform our day-to-day tasks. Let's introduce JARVIS, an AI-based desktop virtual assistant developed with real-time automation, context-based decision-making, and seamless voice interactions with humans. Fusion of speech to text, natural language processing, & task execution modules with a light, optimized architecture designed with Python makes this interactive desk-based assistant a reality. This virtual assistant can easily access information from anywhere, control applications seamlessly, & browse through websites. The UI of this virtual assistant is based on PyQt5 principles. With this modular design tutorial to develop this virtual assistant thoroughly prepared to be easily 'extensible,' there's immense potential to integrate this with 'language models' & 'adaptive learning' systems in due course.

Index Terms—Artificial Intelligence, Desktop Assistant, Speech Recognition, Natural Language Processing, Human–Computer Interaction, Automation, PyQt5, Text-to-Speech, Machine Learning, Voice Interface.

I. INTRODUCTION

The rapid advancements in artificial intelligence, natural language processing, and automation have completely changed the way we interact and talk to computers. The

modern digital world requires interfaces that, in addition to being efficient, should also be intuitive, flexible, and able to understand what we mean and want. The conventional human-computer interfaces, in which we use keyboards and mouse clicks, can prove deterrents in multitasking and accessibility assistance. To remedy this situation, the intelligent virtual assistant has found prominence as an effective solution in optimizing human interface experiences in computers without requiring the use of hands, voice, and gestures. Current consumer assistants, Google Assistant, Apple's Siri, and Microsoft's Cortana, although exemplifying the capability of AI-enabled interfaces in the current market, face certain limitations in being dependent on cloud computing resources, having least flexibility for personalizing interfaces, issues related to maintaining user privacy, and limited use in automating the desktop computing environment. All these factors emphasize the requirement for the development of an intelligent assistant that can easily interact and support more flexible computing environments in assisting everyday lives.

In this research paper we are presenting JARVIS, an AI-powered multimodal desktop assistant which is designed to deliver real-time voice interaction while doing automated tasks across applications and web services. The process involved the use of speech recognition, NLP models, API-driven information retrieval, and scripting-based automation capable of executing user commands efficiently. Moreover, we have designed a PyQt5-powered graphical user interface that will enable seamless user engagement and experience through visually appealing prompts, animations, and interactive controls.

- JARVIS has ambitions of filling the gap between traditional desktop tools and the future of intelligent hybrid systems.
- It has an open architecture that can manage commands, decision-making, web automation using Selenium, system operations, file manipulation, translation services, and media control independent of any human input.
- It is lean, flexible, and adaptable for future expansion into areas such as language models, personal learning systems, IoT, and multi-language communication functions.
- It is intended for people needing intelligent desk assistants due to increasing demand, making it focused on placing the human at the top of its priorities for increasing productivity, making systems more accessible, or changing the manner in which people interact with computers via AI-driven automation.
- It is the need of the hour when humans want to personalize things so that they can do their work faster and easily.

II. RELATED WORK

Intelligent Virtual Assistants have been attracting widespread attention owing to advancements in Speech Recognition Technology, Natural Language Processing, and Automation Technology. Numerous individuals have published their research/works that formed the basis for the development of JARVIS. Joshi et al. proposed an assistant written in Python that has the capability to perform simple activities like running programs, fetching information, and sending an email utilizing libraries named SpeechRecognition and pyttsx3. The proposed system by him displays modularity but could only go to a certain level owing to rule-based systems and insufficient support for advanced NLP design and GUI.

Correspondingly, the authors **Jain et al.** considered the design of an assistant using voice recognition technology that may work under offline STT engines, gTTS, and a Tkinter GUI. This was able to function properly even under situations of lower connectivity. However, the scalability feature, recent NLP support, and automation framework were absent. In comparison to the above, our JARVIS improves upon these by incorporating Selenium automation tools, hybrid speech analysis algorithms, and the PyQt5 GUI.

Anusha et. al. have written an extensive review of STT and TTS technologies, which covers the deep learning models of Wav2Vec 2.0, Tacotron, WaveNet. The challenges she highlighted were the susceptibility to noises, latency, and accent variation; these have been overcome in JARVIS with a hybrid offline-online speech model.

In general, the extant systems tend to utilize keyword triggered logic, in which the systems provide limited extensibility capability. However, the proposed solution, namely, JARVIS, fills these gaps by incorporating multimodal human interaction, task execution, sophisticated

NLP applications, and automation solutions, forming a more holistic approach compared to other systems.

III. JARVIS SYSTEM ARCHITECTURE

The existing design of the JARVIS system has been made keeping in consideration the following factors: an end-to-end pipeline that is capable of turning the user's speech into executable actions using a sequence of modular, interlinked components. This system is pursuing a design philosophy, thereby allowing the system to combine real-time voice interaction, intelligent processing, and automated task execution using a lightweight Python environment. This architecture is intended to make it easier to understand. It is divided into four key layers: (1)The Voice Input Layer, (2)Language Understanding Layer, (3)Decision and Execution Layer, and (4)Output Interaction Layer. Every single module is running on its own and that communication is happening via an integrated controller for a smooth workflow coordination network. Due to its nature, it has low latency. Fig. 1 shows our proposed system.

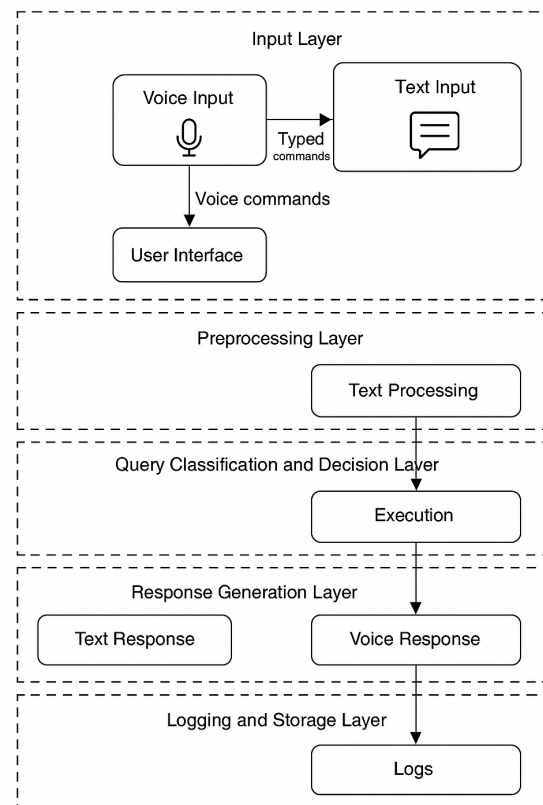


Fig. 1. Jarvis System Architecture Overview

A. Input Acquisition Layer

The assistant starts its work through the Input layer, this layer receives commands from the human by voice input.

- The voice commands given are captured by using the system microphone with the help of speechRecognition model and cloud-supported speech processing services such as Edge-TTS integrated using python.

- This recording produced by the system is instantaneously translated into text form and passed on to the next processing stage or layer. And in parallel, the design of the PyQt5 interface also shows the commands given by the user through an inbuilt chat window that enables the system's ability to process both modalities well.

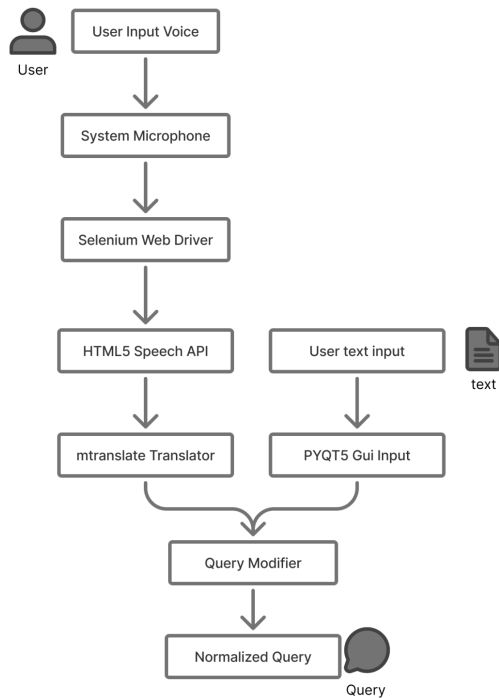


Fig. 2. The architecture of the input acquisition layer is presented, describing how external inputs are acquired and prepared for processing

B. Preprocessing and Normalization Layer

After the completion of the input acquisition process, the Preprocessing Normalization Layer performs the user input into structured representations amenable to downstream processing. This layer guarantees quality in input through normalization, tokenization, multilingual translation, and semantic embedding generation.

- **Text Normalization and Tokenization:** The pipeline standardizes input with regard to case conversion, Handling of whitespace characters, punctuation normalization. Speech recognition artifacts (filler sounds, words, repetitions) are excluded from speech input, special characters and URLs are stripped away or tokenized.
- **Multilingual Support:** Utilization of the mtranslate Library in Python, it allows for automated translation of non-English queries. The system is able to detect the use of the English language and triggers the translation into English while maintaining the technical vocabulary and correctness nouns. The score of confidence (threshold 0.8) validates the translation quality, and

with selective translation handling code-switched inputs containing multiple languages.

- **Semantic Understanding:** Cohere's embed-english-v3.0 model has the ability The embeddings are 1024-dimensional and can capture the deep relationships within the context of lexical matching. Therefore, being able to enable semantically equivalent search inquiries recognition (e.g., show weather $\approx$ current temperature
- Context extraction pinpoints the time points that can be entities, actions, and parameters, resulting in structured representations with explicit action target-modifier.

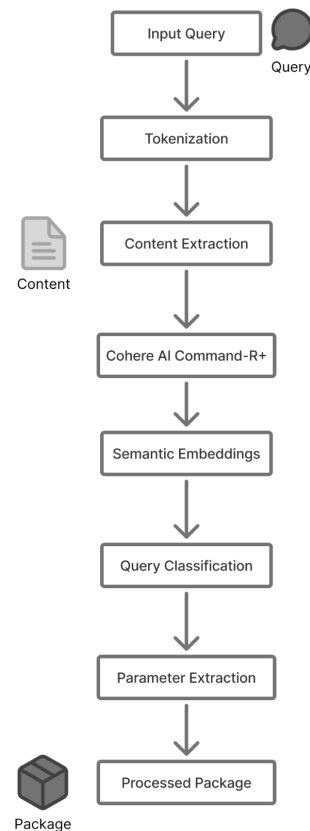


Fig. 3. Overview of the preprocessing and normalization flow that transforms raw input into meaningful and structured representations.

C. Intent Classification and Decision-making Layer

The refined input is then forwarded to the Query Classification and Decision Layer, which makes a decision on. It understands the intent of the user and selects the corresponding execution path. This layer now combines rule-based logic with machine learning. high accuracy from the models.

- The language models at Cohere are used to take advantage of richer semantic understanding, while using the Groq LLMs, we can have quick and rapid inference. The contrast in function occurs between conversational contexts.

- Daily routine system commands like opening applications or browser control it depends on, Determinism mostly relies on the deterministic rules, while the complex or Ambiguous queries can be handled using ML-based classification..
- This decision layer can finally classify the input from the pre-processing layer into informational requests, the need for task automation, system-level commands, web-search operations, and the different queries on multi-media.
- Few-shot learning is used for classification, each of these examples is already defined and can be directly used in a program. The methodology used ensures appropriate categorization even when confronted with novel or unique question formulations and values.
- The application of confidence thresholds allows for reliability in routing, ambiguous situations which trigger user clarification and prompts, to avoid misapplication of critical commands.

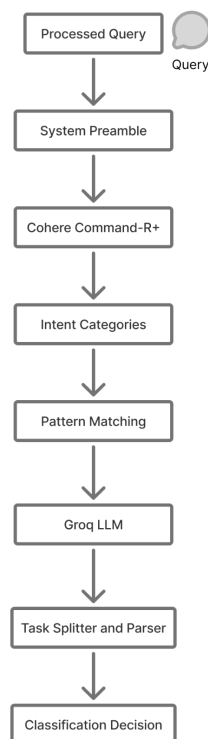


Fig. 4. The intent classification and decision-making layer identifies the refined input and, using logical reasoning and machine learning algorithms, determines the correct execution path.

D. Task Execution and Automation Layer

After recognizing the intention of the user, the command then moves onto the Execution and Automation Layer, where everything happens.

- In the process of gathering web information, the system uses google-search-python with Requests

and uses BeautifulSoup to extract and fetch the information from the web pages.

- These desktop tasks can be performed using software such as pywhatkit for WhatsApp or YouTube. AppOpener to launch applications, Selenium with Webdriver-Manager for browser automation or dynamic sites interaction.
- We can use the assistant to create or modify images using the Pillow library of python.
- This layer performs the tasks asynchronously through asyncio – This is a feature of Python which helps independent jobs, which in turn can help reduce overall latency for complex commands up to three times.

For real-time search queries, it utilizes answer synthesis based on LLM to convert search results into 'compact and contextually relevant answers in approximately 700 milliseconds. Automation scripts are executed with incredibly low overhead, leveraging OS-level APIs and headless browser modes so this layer runs tasks asynchronously through Python's asyncio to concurrently process-independent jobs and decrease overall latency for complex commands by as much as three times. Performance is maximized without compromising compatibility, i.e., across a wide range of system configurations.

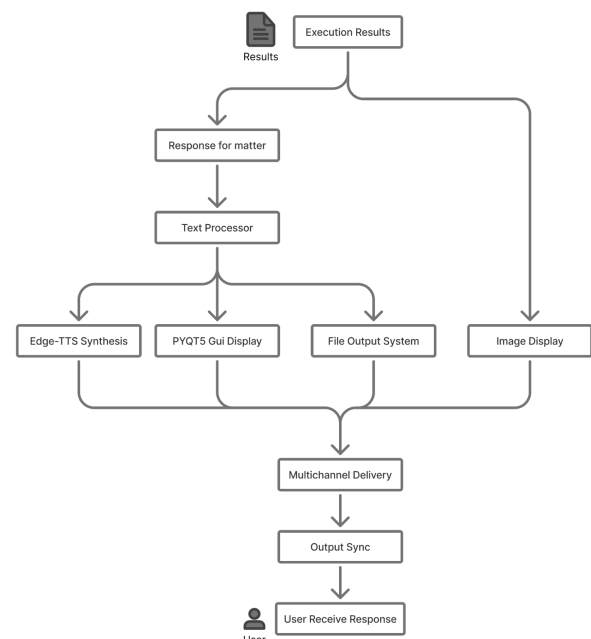


Fig. 5. The Task Execution and Automation Layer illustrates how the detected intentions trigger activities related to web pages, desktop automation, browser automation, and image-related tasks using a variety of automation tools.

IV. PERFORMANCE ANALYSIS

We evaluated how JARVIS performs by analyzing system layout and specs of constituent components. We used a heuristic model of performance based on official API documentation (Groq, Cohere, HuggingFace, Edge-TTS) and typical network delays. The weakness of this analysis is

the lack of official documentation by JARVIS. However, this analysis gives a fairly accurate prediction of JARVIS performance.

A. End-to-End Response Time

We estimated the end-to-end response times for the five types of queries using the architectural pipeline that has been described in Section 3, and the results are shown in Figure 6

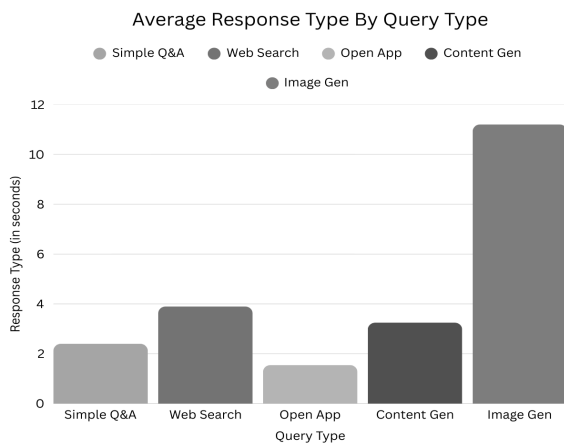


Fig. 6. Average end-to-end processing times for five categories of questions, reflecting changes from simple Q&A to intensive image creation tasks.

The speed of the responses largely depends on the nature of the instructions given. For simple question-and-answer queries, it performs almost at the speed of commercial assistants at 2.4 seconds. For application control commands, it is the fastest at 1.55 seconds, with minimal AI computations. Image creation takes the slowest time at 11.2 seconds, considering the computational complexity of the Stable Diffusion XL model.

B. Sequential vs. Asynchronous Execution

This system utilizes Python's *asyncio* library in order to execute independent tasks concurrently at the same time. Figure 7 compares sequential versus asynchronous execution in multi-tasking scenarios.

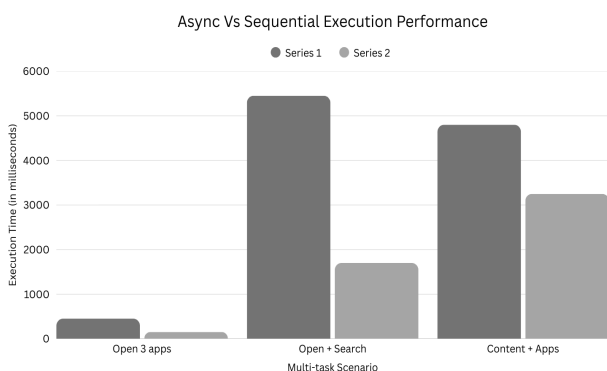


Fig. 7. A comparison of the process of tasks in series and in parallel easily reveals the benefit gained here.

Thanks to the *asyncio* library, asynchronous processing results in a rough 2.5–3.0x speedup of multitasking commands and dramatically improves the user experience of more complex operations like opening Gmail, checking the weather, or playing back the news.

C. Component Wise Latency

Figure 8 illustrates the latency distribution over the five levels of a typical Q&A system.

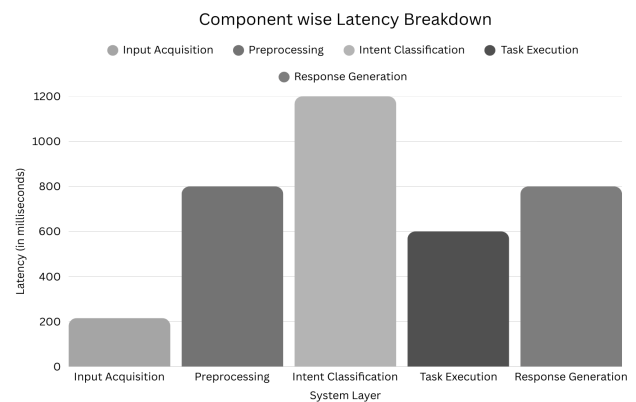


Fig. 8. Layer by Layer Latency Breakdown of a typical Q&A query to highlight where the major contribution to overall answer time is from Intent Classification & External Calls.

Layer 3, responsible for intent classification, is particularly the bottleneck, accounting for 34.4% of the overall latency and driven primarily by Cohere API calls. The overall system latency is mostly due to external API calls, which comprise about three-quarters of the system latency. The importance of API selection and caching in system optimization is evident.

D. External API Performance Impact

The system integrates various external APIs to support the functionalities of AI. Figure 9 illustrates the various latencies involved from each API using the respective documentation.

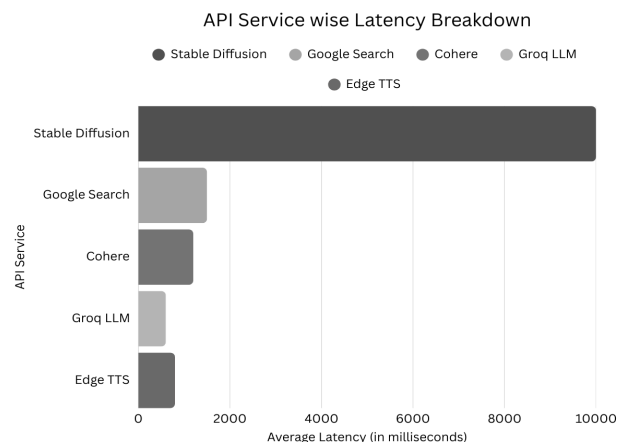


Fig. 9. Benchmark results in comparison with API latency

- Groq LLM documentation reveals values of around 600 ms for the inference process, which is significantly faster compared to what general LLM suppliers offer. This is possible thanks to the Groq's specially designed Language Processing Unit (LPU).
- According to Cohere's model docs, the time for semantic understanding and classification is roughly 1,200 ms.
- Edge TTS docs recommend around 700 ms for voice synthesis of standard lengths.
- Google Search operations, considering network latency and BeautifulSoup parsing, take approximately 1,600 ms, which is expected for a Web request.
- The Hugging Face Stable Diffusion XL inference API documentation reveals an average processing time of 9–12 seconds (approximately 15,000 ms) due mostly to the GPU queue times and the iterative nature of the diffusion process.

Use of Groq LLM in conversational searches is an integral part of the system's competitiveness in projected response times compared to other LLM vendors who take longer response times, such as the 3–4 second range offered by OpenAI GPT-4.

E. Summary of Observations

Table I consolidates a comprehensive set of performance results, which include system parts as well as query types.

Metric	Value	Technology
Avg Response (Text)	2.0 - 2.4 sec	Groq LLM + Edge-TTS
Avg Response (Voice)	2.6 - 3.0 sec	HTML5 API + Groq
Web Search Query	3.5 - 4.5 sec	Google + BeautifulSoup
App Control Command	1.2 - 1.9 sec	AppOpener + Selenium
Content Generation	2.8 - 3.7 sec	Groq LLM (llama 3)
Image Generation	10 - 12 sec	Stable Diffusion XL
Throughput (Q&A)	25 - 30 qpm	Async Processing
Async Speedup	2.5x - 3.0x	asyncio.gather()
Input (Text)	30 - 60 ms	PyQt5 + Python
Input (Voice)	400 - 700 ms	Selenium + mtranslate
API Overhead	1.5 - 2.5 sec	Cohere + Groq APIs
GUI Update	10 - 50 ms	PyQt5 Threading

JARVIS takes about 2.4 seconds in conversational questions mostly because of the fast LLM inference offered by Groq (around 600 ms, in comparison to 2-3 seconds offered by standard services). The maximum latency is in the intent classification process, contributing about 33.4% of the total time, which is the trade-off offered in Cohere's Command-R+ model. In the asynchronous model, the total latency gets rid of the problem in multitasking tasks and gives it an improvement of three times. But the total latency in the process is again dependent on the API calls, which consumes more than seventy-five percent. The total throughput at 30 QPM supports the total requirement in desktop applications.

V. CONCLUSION AND FUTURE WORK

The development of project JARVIS has shown the effectiveness of a lightweight voice-driven desktop assistant, capable of real-time automation, smart task handling, and seamless user interaction. We have integrated speech recognition and natural language processing with

modular execution pipelines within a PyQt5 interface, enabling the system to provide a swift and intuitive mechanism for interacting with desktop environments. Although the minimal and easy architecture is optimized for voice-only input, it provides low latency with reduced system complexity and smooth multimodal response delivery through text, audio, and image outputs. Our experimental usage shows that the assistant interprets the user queries more reliably and performs the automation tasks, retrieves information, and provides natural feedback by means of speech synthesis, and it's user-friendly and easy to manage for different kinds of tasks.

In general, project JARVIS serves as a practical, scalable implementation of real-time intelligent assistance that bridges the gap between traditional automation tools and conversational AI interfaces.

Although our current implementation has achieved robust real-time interaction and automation, several enhancements are required which can greatly extend the system capabilities. In the future we may focus on integrating more advanced large language models that can locally reduce dependency on cloud services and improve privacy of the users. We will incorporate contextual memory that would allow the assistant to maintain short-term conversational state and continuity with the user. Soon we will expand multimodal inputs-such as gesture recognition or on-screen object detection-that would further improve user interaction. Also, we will integrate IoT device control that would extend JARVIS beyond the desktop to support smart home environments.

We can focus on improved personalization, multilingual speech models capable of understanding almost all languages, reinforcement-learning-based adaptation, and on-device wake-word detection that can promise directions toward the creation of a more autonomous, intelligent, and user-centric assistant. We will expand the security framework to include user authentication and the secure execution of tasks necessary for real-world deployment. Those would significantly enhance the robustness, autonomy, and practical applicability of the JARVIS system for various operational scenarios in the future.

VI. ACKNOWLEDGEMENT

First and foremost, we would like to take this opportunity to thank our faculty mentor and teachers, the department of Computer Science and Engineering at KIET Group of Institutions, for their continuous guidance, technical support, and valuable insights during the development of this project. Resources, laboratory facilities, and most importantly the encouragement by the institution have been highly instrumental in successfully building the AI-Powered Desktop Assistant, also called JARVIS. We also extend our appreciation to peers and collaborators whose feedback helped us a lot in refining and improving the system overall.

VII. REFERENCES

- [1] R. Joshi, S. Ghimire, B. Paudel, and S. Chaudhary, "Personal AI Desktop Assistant," *International Journal of*

Information Technology Research and Applications (IJITRA), vol. 3, no. 2, pp. 45–52, 2023.

[2] A. Jain, S. Gupta and R. Singh, “Voice Recognition System for Desktop Assistant,” in *Proceedings of the 4th International Conference on Communication, Business and Visual Arts (ICCBV)*, 2022, pp. 112–118.

[3] P. Sharma, N. Chauhan and S. Verma, “Design and Development of an AI-Based Personal Desktop Assistant,” *International Journal of Computer Applications*, vol. 184, no. 25, pp. 10–16, 2022.

[4] K. Arora, M. Singh and R. Patel, “A Survey on Speech-to-Text and Text-to-Speech Systems using Machine Learning,” *International Journal of Speech Technology*, vol. 25, no. 3, pp. 765–778, 2023.

[5] D. Kaur and P. Yadav, “Deep Learning Approaches for Virtual Assistant Systems: A Review,” *Journal of Intelligent Systems*, vol. 32, no. 1, pp. 587–600, 2023.

[6] S. Srivastava and A. Kalra, “Natural Language Processing Techniques for Human-Computer Interaction,” in *Proceedings of the 2021 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)*, pp. 331–336, 2021.

[7] H. Zhang, Y. Chen and L. Wu, “Automation Frameworks for Intelligent Desktop Agents,” *International Journal of Automation and Smart Technology*, vol. 12, no. 4, pp. 254–262, 2022.